This week we want to understand how quickly our approximations converge, how we can efficiently construct and differentiate Chebyshev interpolants and why we might eventually abandon one global polynomial in favor of several small local polynomials

From Interpolation to Convergence

- $p_m(x) = \sum_{k=-m}^{m} c_k e^{2\pi i k x}$ we can approximate periodic functions using trigonometric polynomials.

We still want to know:

→ What happens when we increase the number of interpolation points?

→ How fast does the error decrease?

→ Will the derivative of our approx. also converge to the true derivative?

Convergence Analysis

Let p_n be an approx. of function f.

We can measure its error using different norms. Two particularly useful choices are:

- $E_\infty(n) = \|f - p\|_\infty = \max_{x \in [a,b]} |f(x) - p_n(x)|$ → worst error over the entire interval
- $E_2(n) = \|f - p\|_{L^2(a,b)} = \left(\int_a^b |f(x) - p_n(x)|^2 \, dx \right)^{1/2}$ → accumulated squared error

Reminder: $L^2(0,1)$

$L^2(0,1) = \{ f : (0,1) \to \mathbb{C} : \|f\|_{L^2(0,1)} < \infty \}$, so functions whose squared magnitude is integrable: $\int_0^1 |f(t)|^2 \, dt < \infty$.

⇒ We introduced this because fuctions in L^2 behave like vectors in vector spaces. So we defined scalar product and orthogonality

Why do we need both?

Imagine 2 approx. methods are being compared.

The first gives a small error nearly everywhere, but produces narrow, large spikes near an endpoint → small L^2

The second produces a slightly large error throughout the interval, but no spikes → small ∞-norm

Numerical approximation of the error

In CodeExpert we generally need to approximate these norms using a fine reference grid:

$$t_j = \frac{j}{N} \qquad j = 0, \dots, N-1$$

⇒ Thus: $$E_2(n) = \left(\frac{1}{N} \sum_{j=0}^{N-1} |f(t_j) - p_n(t_j)|^2 \right)^{1/2}$$

```
l2 = np.linalg.norm(v - fv) / np.sqrt(N)
```

$$E_\infty(n) = \max_{j=0,\dots,N-1} |f(t_j) - p_n(t_j)|$$ → could underestimate true error, how?

```
linf = np.max(np.abs(v-fv))
linf = np.linalg.norm(v - fv, ord = np.inf)
```

N.. how finely we evaluate the error, usually larger N gives a better numerical approximation

Convergence Rates

Algebraic: $E(n) = O\left(\frac{1}{n^p}\right) \quad p > 0$

Exponential: $E(n) = O(q^n) \quad 0 \leq q < 1$

Example:

Suppose $E(n) = Cn^{-2}$, we double n: $\frac{E(2n)}{E(n)} = \frac{C(2n)^{-2}}{Cn^{-2}} = \frac{1}{4}$

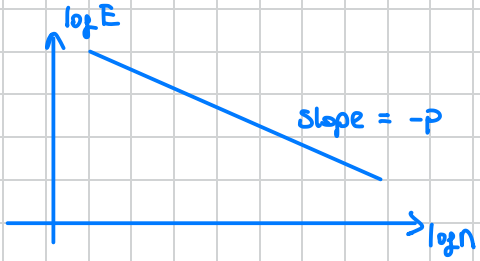
Suppose $E(n) = Cq^n$, we double n: $\frac{E(2n)}{E(n)} = \frac{Cq^{2n}}{Cq^n} = q^n$ → take $q = 1/2$

Reminder: How to recognize these in plots?

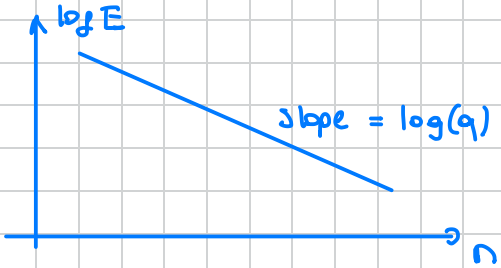
$E(n) = Cn^{-p} \iff \log E(n) = \log C - p \log n$

$E(n) = Cq^{n} \iff \log E(n) = \log C + n \log p$

Algebraic:



Exponential:



Why does smoothness determine convergence?

We have seen that $\hat{f}(k) = \int_0^1 f(x) e^{-2\pi i k} dx$ and we saw that we keep only a finite number of frequencies: $S_m f(x) = \sum_{k=-m}^{m} \hat{f}(x) e^{2\pi i x}$

→ Here m is the highest frequency retained. The smaller the discarded coeffs, the smaller the approx. error.

- How does smoothness affect these coeffs.

↳ recall Fourier differentiation property: $\underbrace{\hat{f'}(x)}_{\text{coeffs of } f'} = 2\pi i k \hat{f}(x)$

Do this r times $\Rightarrow \widehat{f^{(r)}}(x) = (2\pi i k)^r \hat{f}(x)$

$$\iff \hat{f}(x) = \frac{\widehat{f^{(r)}}(x)}{(2\pi i k)^r} \qquad k \neq 0$$

This tells us that if f has r sufficiently regular periodic derivatives, we can bound its Fourier coeffs:

$$|\hat{f}(x)| \leq \frac{\| f^{(r)} \|_{L^1}}{(2\pi |k|)^r} = O(|k|^{-r})$$

So smoothness means we can use higher r values in our theoretical analysis, and thus we have faster decaying coefficients across increasing k.

Understanding Fourier Transform Conventions

Convention 1 – Positive Exponential

$$f(x) \approx \sum_k c_k e^{2\pi i k x}$$

for calculating the coeffs from n equally spaced samples $y_j = f(j/n)$ we use

$$c_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j e^{-2\pi i k j / n}$$

Notice the opposite signs: – for calculating coeffs, + for reconstruction

In NumPy:

```
c = np.fft.fft(y) / n
y = np.fft.ifft(c) * n
```

For differentiation: $\frac{d}{dx} e^{2\pi i k x} = 2\pi i k e^{2\pi i k x}$

Convention 2 – Negative Exponential

$$f(x) \approx \sum_k c_k e^{-2\pi i k x}$$

In NumPy:

```
c = np.fft.ifft(y)
y = np.fft.fft(c)
```

→ no division by n as ifft in NumPy already does this!

For differentiation: $\frac{d}{dx} e^{2\pi i k x} = -2\pi i k e^{2\pi i k x}$

Code Expert: Konvergenz Trigonometrischer Interpolation

Here we compare 4 periodic function v(x), n(x), a(x) and b(x) with different smoothness properties.

1. v(x) is limited regarding smoothness
2. n(x) is smooth but non-analytic
3. a(x) is smooth and analytic → a function that can be represented by its Taylor series around every point
4. b(x) is band-limited
   - a function containing only finitely many nonzero Fourier coeffs
   - Once resolved, Fourier interp reconstructs it exactly

Discussion

- Smoothness and being band-limited is good for convergence. But being analytic also helps.
- Increasing n → more accuracy, but also more computation

Code Expert: Chebyshev Interpolation mit der baryzentrischen Formel

Chebyshev Interpolation : Focussed Reminder

Runge Phenomenon: Interpolating at equally spaced nodes with high-degree polynomial can produce large oscillations near the endpoints.

- $T_n(x) = \cos(\arccos x) \qquad x \in [-1, 1]$

  ↳ recurrence: $T_{n+1}(x) = 2x\,T_n(x) - T_{n-1}(x)$

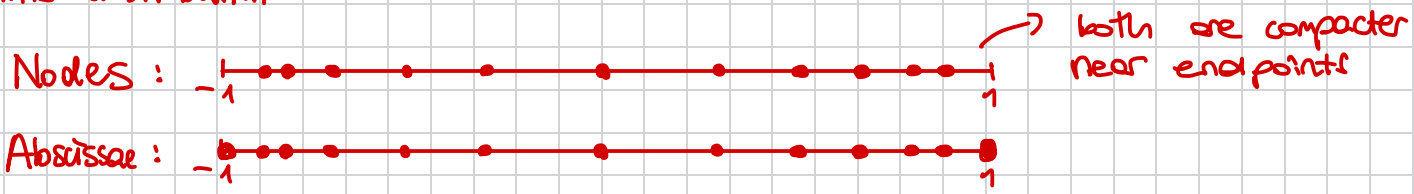
Chebyshev Nodes, which are zeros of T_{n+1}:

$$x_j = \cos\left(\frac{(2j+1)\pi}{2(n+1)}\right)$$

Chebyshev Abscissae, corresponding to extrema of T_n:

$$x_j = \cos\left(\frac{j\pi}{n}\right) \quad \text{where} \quad t_j = \frac{j}{n}$$

❗ Points distribution:



Revisiting barycentric interpolation

Recall that the lagrange interpolating polynomial is $p_n(x) = \sum_{j=0}^{n} y_j L_j(x)$

we have derived barycentric interpolation as:

$$p_n(x) = \frac{\sum_{j=0}^{n} \frac{\lambda_j y_j}{x - x_j}}{\sum_{j=0}^{n} \frac{\lambda_j}{x - x_j}}$$

We have said that computing $\lambda_j = \prod_{j \neq k} \frac{1}{x_j - x_k}$ for all nodes generally require to much time, so we avoid this.

- Instead we use Chebyshev abscissae and weights than have a really simple form.

↓ this has a long derivation

$\Rightarrow \lambda_j = (-1)^j \delta_j$ with $\delta_j \begin{cases} 1/2 & j=0 \text{ or } j=n \rightarrow \text{endpoints} \\ 1 & \text{otherwise} \end{cases}$

```
n = 4
j = np.arange(n+1)
x = np.cos(np.pi * j / n)
w = (-1) ** j
w[0] *= 0.5
w[1] *= 0.5
```

Why are the weights unchanged when we change intervals?

So for our Chebyshev points lie in $[-1, 1]$, we want to interp. on interval $[a, b]$:

$$x_j = a + \frac{b-a}{2}(t_j + 1)$$

under the transformation $\lambda_j = \dfrac{1}{\prod_{k \neq j} x_j - x_k}$ look at one term in product:

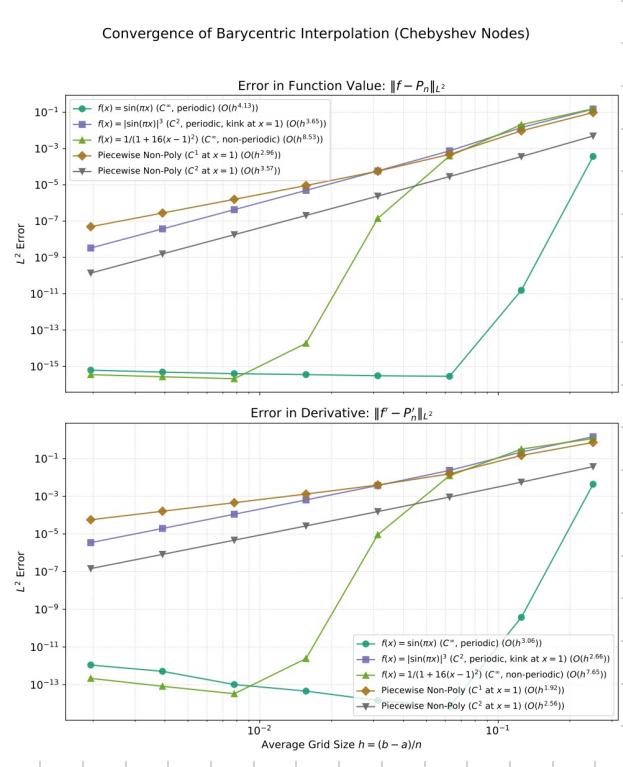
$$x_j - x_k = \frac{b-a}{2}(t_j - t_k)$$

since every λ_j have n such differences, each weight is multiplied by:

$$\left(\frac{2}{b-a}\right)^n$$

But multiply both nominator and denominator with the same nonzero constant term implies our formula is unchanged.

Some simplified Chebyshev weights can be used on any interval $[a, b]$



Coding Tips

- Barycentric Interpolator gives interpolant ready to be used as $\bar{f}(x) = \bar{y}$
- use .derivative, do not use np.gradient as stated in the task
- you need the transformation from $[-1,1]$ to $[a,b]$
- Barycentric Interpolator needs x_j (above), $f(x_j) = y_j$ and weights.

Discussion: Why differentiating an interpolant is not the same as finite differences?

Here we used both $p_n(x_j) = f(x_j)$ and $p_n'(x) \approx f'(x)$

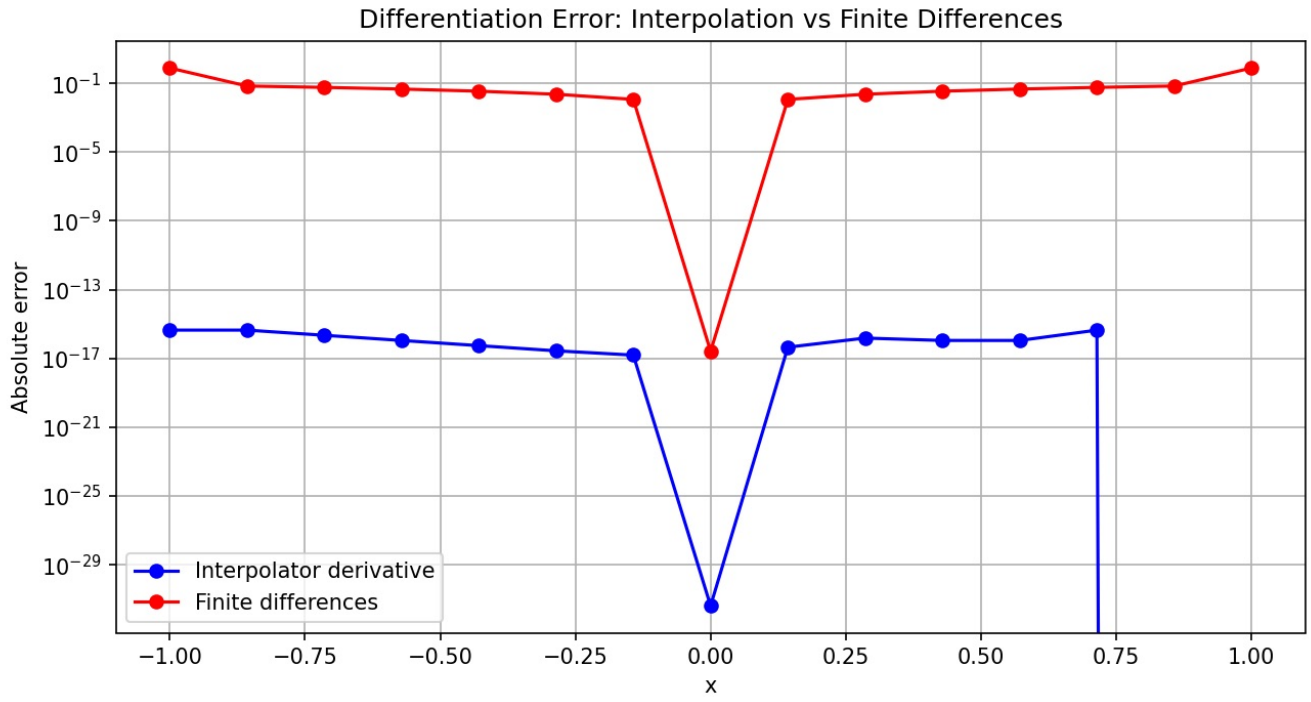
we differentiate the polynomial representation of function values

Recall session one – Central finite differences

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_{j-1})}{2h} \quad \text{error is } O(h^2)$$

Here np.gradient implements the above method, it does not compute the derivative but approximates it. → We don't want this error!

→ Code Example 1



Code Expert : Schnelle Chebyshev-Interpolation (DCT-I)

The Connection Between Fourier and Chebyshev Interpolation

Goal: Approximate $f(x)$

Session 1 → Chebyshev Interpolation expresses a function as a combination of Chebyshev polynomials

Session 3 → periodic functions expressed as a combination of oscillatory functions

remember $x \in [-1,1]$ →

$$T_k(x) = \cos(k \arccos x), \text{ then introduce } x = \cos\theta$$

$$\Rightarrow T_k(x) = \cos(k\theta) \quad (A)$$

Suppose

$$p_n(x) = \sum_{k=0}^{n} c_k T_k(x)$$

$$p_n(\cos\theta) = \sum_{k=0}^{n} c_k \cos(k\theta) \quad (A)$$

We define $g(\theta) = f(\cos\theta)$, then approximating $f(x)$ using Chebyshev polynomials is equivalent to approximating the transformed function $g(\theta)$ using cosine functions.

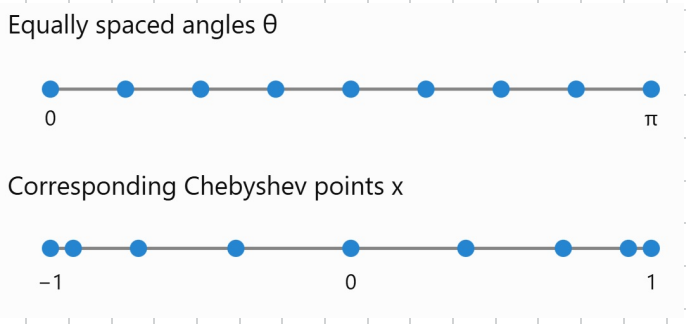
Chebyshev points become equally spaced?

$x_j = \cos\left(\frac{j\pi}{n}\right)$ $j = 0, \dots, n$ do the coordinate transformation

$$\Rightarrow \theta_j = \frac{j\pi}{n} \Rightarrow x_j = \cos\theta_j$$

Example $n = 4$

j	θ_j	x_j
0	0	1
1	$\pi/4$	0.707
2	$\pi/2$	0
3	$3\pi/4$	-0.707
4	π	-1



Take (A) again:

$$p_n(\cos\theta) = \sum_{k=0}^{n} c_k \cos(k\theta)$$

At the interpolation nodes $\theta_j = \frac{j\pi}{n}$ we require:

$$p_n(\cos\theta_j) = f(\cos\theta_j) = y_j$$

Substitute:

$$y_j = \sum_{k=0}^{n} c_k \cos\left(\frac{kj\pi}{n}\right)$$

known function samples

unknown coefficients!

Here we need Discrete Cosine Transform (DCT)

Derivation: How is DCT related to FFT

$$\Rightarrow p_n(\cos\theta) = \sum_{k=0}^{n} c_k \cos(k\theta) \quad \text{recall Euler's identities: } \cos(k\theta) = \frac{e^{ik\theta} + e^{-ik\theta}}{2}$$

$$\Leftrightarrow p_n(\cos\theta) = \sum_{k=0}^{n} c_k \frac{e^{ik\theta} + e^{-ik\theta}}{2} = c_0 + \sum_{k=1}^{n} \frac{c_k}{2} e^{ik\theta} + \sum_{k=1}^{n} \frac{c_k}{2} e^{-ik\theta}$$

Notice: We know have a Fourier representation with + / − frequencies

Why does only cosine remain?

- $g(\theta) = f(\cos\theta)$, since cosine is even ($\cos(-\theta) = \cos(\theta)$)

we get:

$$g(-\theta) = f(\cos -\theta) = f(\cos\theta) = g(\theta) \Rightarrow g(-\theta) = g(\theta)$$

→ also even

Lets express $g(\theta) \sim a_0 + \sum_{k=1}^{\infty} [a_k \cos(k\theta) + b_k \sin(k\theta)]$

Recall Session 3: $b_k = \frac{1}{\pi} \int_{-\pi}^{\pi} g(\theta) \sin(k\theta)\, d\theta = 0$, because an

→ $f(-x) = -f(x)$

even function multiplied by an odd function is odd, and its integral over a symmetric interval is zero.

$$\Rightarrow f(\theta) \sim a_0 + \sum_{k=1}^{\infty} a_k \cos(k\theta)$$

DCT-I convention used by SciPy (★)

$$c_k = \begin{cases} D_k / (2n) & , \quad k = 0 \text{ or } k = 2n \\ D_k / n & , \quad 1 \leq k \leq n-1 \end{cases}$$

Extra: Computing Chebyshev coeffs by a direct matrix-vector mult would require $O(n^2)$ operations. DCT costs $O(n \log n)$

Coding Tips

- Normalize using (★)
- You have already implemented clenshaw
- In the end expect a plot with X and ● corresponding to each other.

Code Expert 3: Chebyshev Interp. mit DFT und Clenshaw

Given $p_n(x) = \sum_{k=0}^{n} c_k T_k(x)$ its derivative is $p_n'(x) = \sum_{k=0}^{n-1} b_k T_k(x)$

Our task is to calculate b_k's from c_k's.

Additional Information

Recall $T_n(x) = \cos(n \arccos x)$ and $U_n(x) = \frac{\sin((n+1)\arccos x)}{\sqrt{1-x^2}}$

⇒ Now differentiate T_n:

$$T_n'(x) = -\sin(n \arccos x) \cdot (\arccos x)' \cdot n$$

$$= -n \cdot \sin(n \arccos x) \cdot \left(-\frac{1}{\sqrt{1-x^2}}\right)$$

$$= n \cdot \underbrace{\frac{\sin(n \arccos x)}{\sqrt{1-x^2}}}_{U_{n-1}(x)} = n \cdot U_{n-1}(x)$$

- $T_0(x) = 1$, $T_1(x) = x$, $T_2(x) = 2x^2 - 1$, $T_3(x) = 4x^3 - 3x$

⇒ $T_1'(x) = 1 = T_0(x)$

⇒ $T_2'(x) = 4x = 4 T_1(x)$

⇒ $T_3'(x) = 12x^2 - 3 = 6 T_2(x) + 3 T_0(x)$

This hints for a recurrence to calculate b_k's

Reminder $T_{n+1}(x) = 2x T_n(x) - T_{n-1}(x)$

Example to understand the recursion

$$p_4(x) = c_0 T_0(x) + c_1 T_1(x) + c_2 T_2(x) + c_3 T_3(x) + c_4 T_4(x)$$

for $T_4(x) = 8x^4 - 8x^2 + 1$ we have $T_4'(x) = 32x^3 - 16x$

$$= 8 T_3(x) + 8 T_1(x)$$

$$\Rightarrow P_4'(x) = c_1 T_0(x) + c_2 4T_1(x) + c_3 (6T_2(x) + 3T_0(x)) + c_4 (8T_3(x) + 8T_1(x))$$

Group terms with same Chebyshev polynomials:

$$\Rightarrow P_4'(x) = (c_1 + 3c_3) T_0(x) + (4c_2 + 8c_4) T_1(x) + 6c_3 T_2(x) + 8c_4 T_3(x)$$

Hence we get:

$$b_0 = c_1 + 3c_3 \; ; \; b_1 = 4c_2 + 8c_4 \; ; \; b_2 = 6c_3 \; ; \; b_3 = 8c_4$$
$$= 2(2)c_2 + b_3 \qquad = 2(3)c_3 \qquad = 2(4)c_4$$

$$\Rightarrow b_k = b_{k+2} + 2(k+1)c_{k+1}$$

Use b_0 to verify: $b_0 = b_2 + 2c_1 = 6c_3 + 2c_1$

$$\neq 3c_3 + c_1$$

$\Rightarrow$ Therefore we need to divide b_0 by 2 after we use this recursion!

Observe that this is a backward recursion! $\Rightarrow$ Define b_{n-1} and b_n first

Important: Chebyshev polynomials are naturally defined on $[-1, 1]$, so for a general interval we have to transform it accordingly

Let $x \in [a, b]$:

$$t = \frac{2(x-a)}{b-a} - 1 \in [-1, 1] \quad \rightarrow \quad p(x) = \sum_{k=0}^{n} c_k T_k(t(x))$$

$$\frac{dp}{dx} = \frac{dp}{dt} \cdot \frac{dt}{dx} = \frac{2}{b-a} \frac{dp}{dt}$$

to get dp/dx afterwards

Coding Tips

$$\frac{dp}{dx}(x_{eval}) = \frac{2}{b-a} \sum_{k=0}^{n-1} b_k T_k(t(x_{eval}))$$

Introduction: Piecewise Polynomial Interpolation

We always used a single global polynomial defined on entire $[a,b]$ to approximate functions.

→ We have seen that discontinuities / limited smoothness are problematic.

What if we divided the interval into smaller subintervals and construct a low-degree polynomial on each subinterval?

Suppose we have $(N+1)$ data points

$$(x_0, y_0), \dots, (x_N, y_N)$$

with

$$a = x_0 < x_1 < \dots < x_N = b$$

We use the given points as break points and construct a straight line between each pair of neighbouring points:

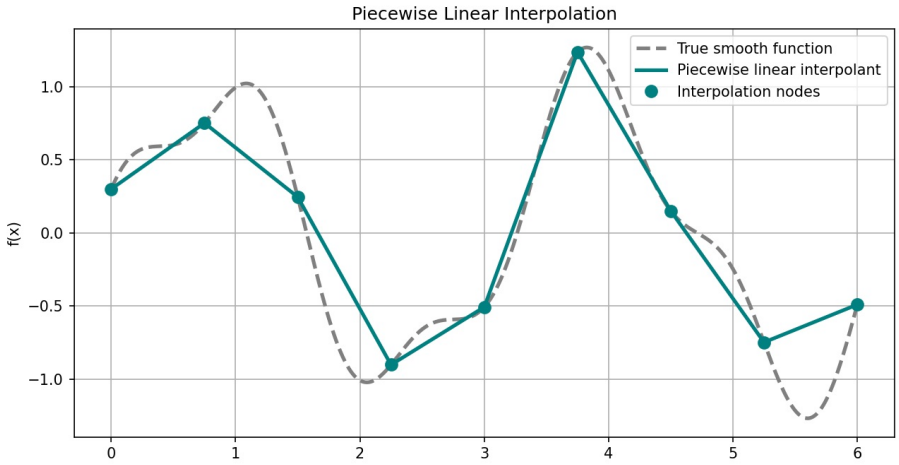
$[x_{j-1}, x_j]$ is a subinterval

$$h_j = x_j - x_{j-1}$$

The corresponding **local** linear interpolant is:

$$s_j(x) = y_{j-1}\frac{x_j - x}{h_j} + y_j\frac{x - x_{j-1}}{h_j}, \quad x \in [x_{j-1}, x_j]$$

$s_j(x_{j-1}) = y_{j-1}$ and $s_j(x_j) = y_j$



- Changing the value y_j only affects the two neighbouring intervals $[x_{j-1}, x_j]$ and $[x_j, x_{j+1}]$ → Locality

Comparison of three interpolation methods:

	Trigonometric	Chebyshev	Piecewise linear
Basis	Fourier modes	Cheby. polynomials	Local line segments
Best for	smoot periodic f	smooth f on finite intervals	general sampled data, incl. less smooth functions
Scope	global	global	local
Main refinement	# Fourier modes	polynomial degree	mesh size h
Advantage	FFT efficiency	good interpolation stability and accuracy	simplicity and locality